

EECE.4520 Microprocessor System II and Embedded System Design

Theodore Skafidas

02022321

Team T

Date of Completion: November 24, 2025

Demonstration Method: in person

Lab 3

Audio-Driven Kinetic Sculpture

I. DESIGN

This section outlines the hardware and software design used to create the audio-driven kinetic sculpture. The system is designed as a standalone embedded application running on an Arduino Mega 2560 microcontroller. This systems utilizes a real-time clock (RTC) for timekeeping and a microphone sensor to drive actuator behavior in real-time.

A. Hardware Design

The hardware design uses an ATmega2560 microcontroller to interface with multiple distinct peripherals using different communication protocols.

- **Actuation:** A DC motor is driven by an L293D H-Bridge motor driver. This isolates the high-current motor load from the microcontroller logic pins. The motor speed is controlled via Pulse Width Modulation (PWM) on Pin 5, while direction is managed via digital logic pins.
- **Timekeeping:** A DS1307 real-time clock (RTC) module is used to maintain the time. It communicates via the I2C protocol (SDA/SCL), connected to the Mega's dedicated I2C pins (20 and 21).
- **Sensing:** An analog sound sensor (microphone) is connected to Analog Pin A0 to detect environmental noise levels. A tactile pushbutton is wired to Pin 6 using the internal pull-up resistor configuration to toggle motor direction.
- **Display:** A 16x2 LCD screen is connected in 4-bit parallel mode (Pins 9-12) to provide visual feedback of the system state, including time, speed, and direction.

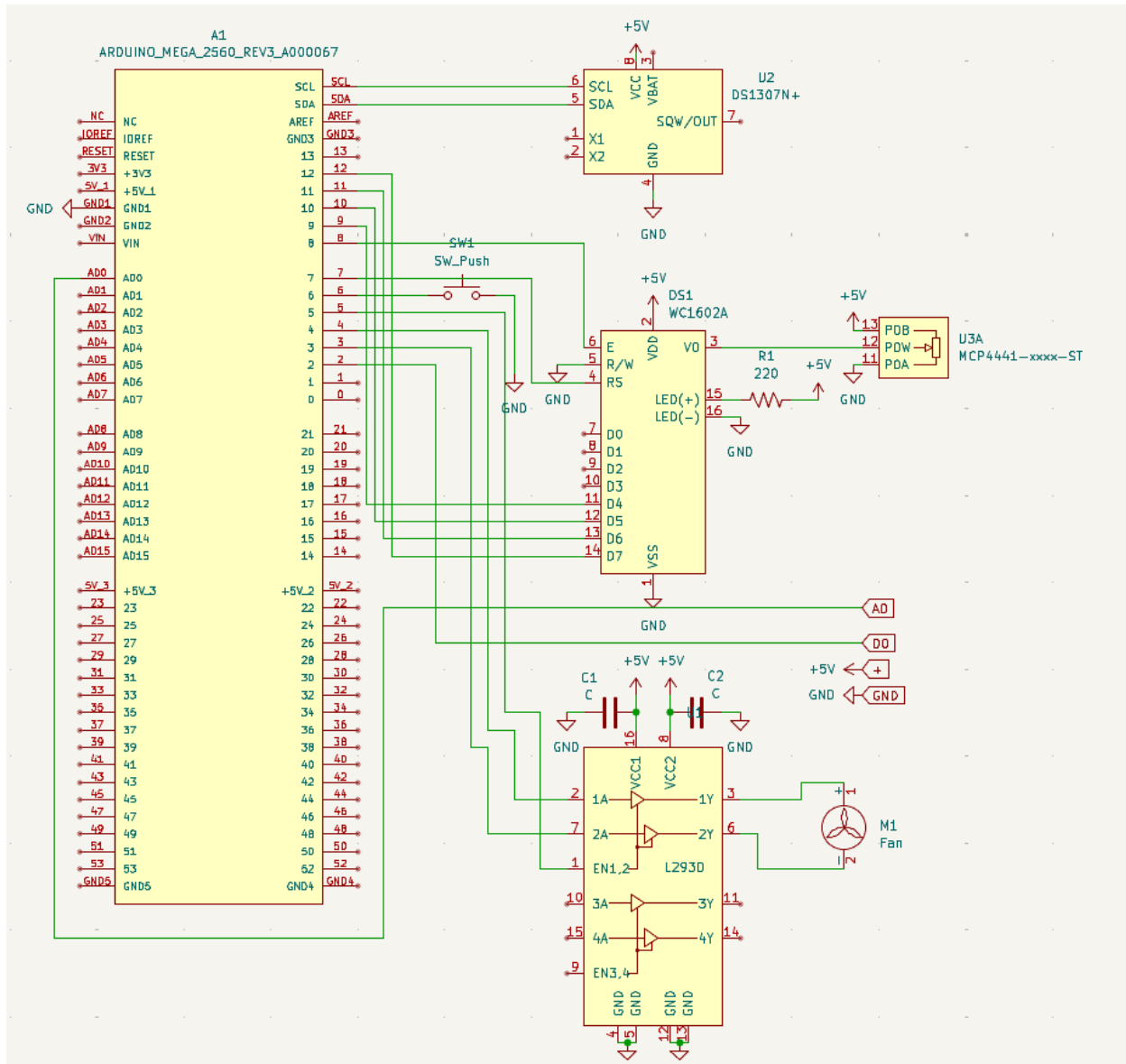


Figure 1 – Kinetic Sculpture Schematic

B. Software Design

The following link contains the complete code used in this lab:

<https://github.com/Tf3tus/Lab3-Audio-Driven-Kinetic-Sculpture.git>

1. Arduino Controller

The software architecture moves away from a simple polling loop and introduces interrupt-driven logic for precise timekeeping, while maintaining a non-blocking "super loop" for sensor processing.

- **Interrupt Service Routine (ISR):** Instead of polling the RTC continuously, the system retrieves the start time once during setup. A hardware timer interrupt (Timer1) is

configured to trigger exactly once per second (1Hz). This ISR increments the global Second, Minute, and Hour variables, ensuring the clock remains accurate regardless of the processing load in the main loop.

- **Audio Sampling Algorithm:** To prevent motor jitter caused by instantaneous noise, the loop() does not use a single analogRead(). Instead, it enters a 50ms non-blocking sampling window using millis(). During this window, the code tracks the signalMax and signalMin to calculate the Peak-to-Peak amplitude (soundLevel).
- **State Machine Logic:** The system operates as a continuous finite state machine.
 - a. **Speed Control:** The soundLevel is compared against three thresholds to set the currentSpeed:
 - i. soundLevel $\geq$ 200: Full speed (PWM 255)
 - ii. soundLevel $\geq$ 100: High speed (PWM 192)
 - iii. soundLevel $\geq$ 45: Low speed (PWM 128)
 - iv. Below 45: Motor off (PWM 0)
 - b. **Direction Control:** The pushbutton toggles a Boolean flag isCW. Depending on this state, the code swaps the logic levels of DIRA and DIRB to reverse rotation.

The following flowchart illustrates the program logic for the Arduino Controller:

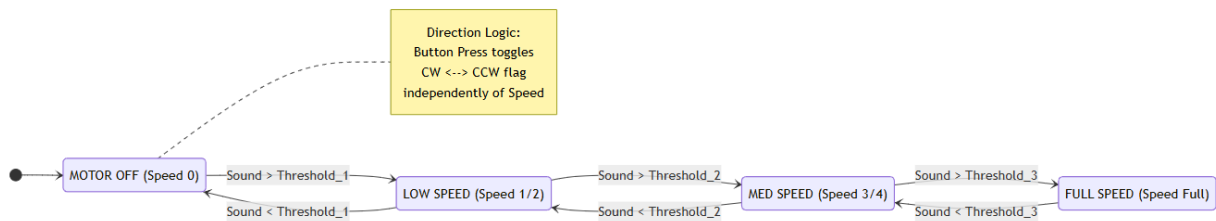


Figure 2 – System Flowchart

C. Results

The kinetic sculpture was successfully implemented and met all functional requirements. The system correctly integrates I2C, Analog, and Digital subsystems. The figure below shows the completed hardware setup.

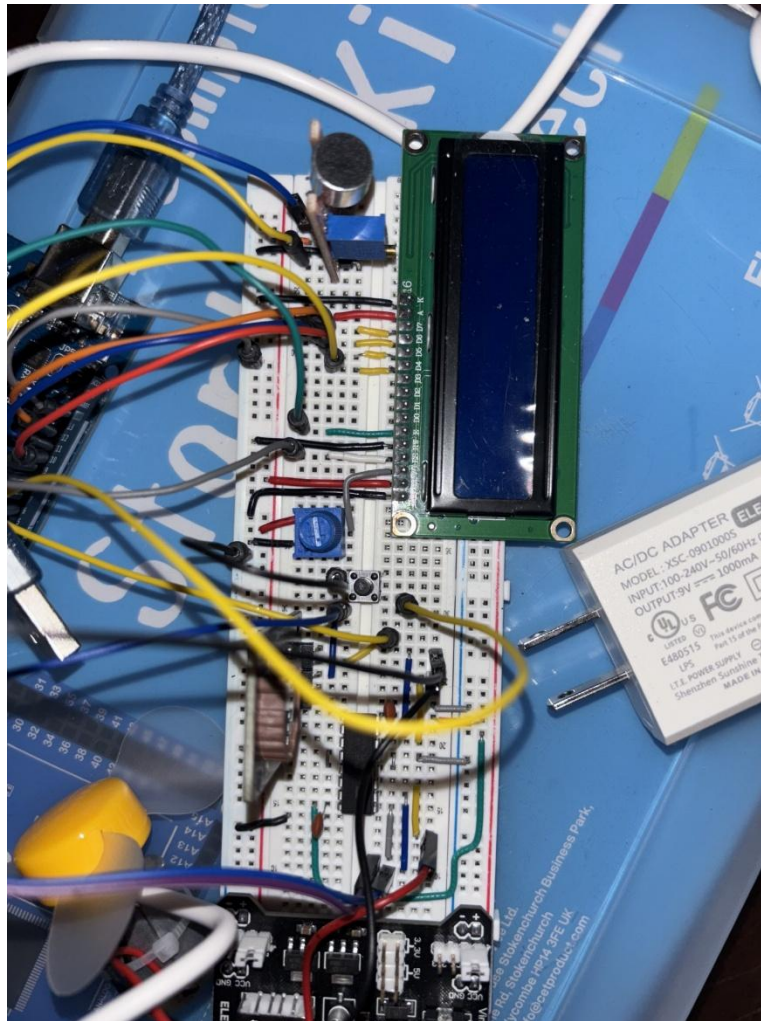


Figure 4 – Physical Sculpture

The following results correspond to the tasks performed by the controller.

1. *Time Keeping*

The Timer1 Interrupt successfully maintained accurate time, incrementing the LCD clock display every second without blocking sensor readings.

2. *Sound Reactivity*

The 50ms sampling window effectively filtered audio noise. The motor transitions smoothly between "Off", "1/2" (128 PWM), "3/4" (192 PWM), and "Full" (255 PWM) speeds.

3. *Visual Feedback*

The LCD correctly displays the time (HH:MM:SS), the mapped speed string (e.g. "Full "), and the direction ("C " or "CC") with no flickering.

4. *Direction Control*

The pushbutton reliably toggles the isCW flag, switching the motor direction between Clockwise and Counter-Clockwise without causing short circuits.

II. PROBLEMS

During the development of the sculpture, several integration challenges were encountered. This section documents the key problems and the solutions applied.

A. I2C Communication Failure

Problem: During the initial setup of the DS1307 RTC, the Serial Monitor returned "Unknown Device" errors.

Solution: The issue was identified as a voltage mismatch. The VCC pin of the DS1307 was initially connected to the Arduino's 3.3V rail. The DS1307 requires 5V logic levels to communicate effectively over I2C. Moving the VCC wire to the 5V rail resolved the communication error.

B. LCD Character "Ghosting"

Problem: When toggling the motor direction from Counter-Clockwise ("CC") to Clockwise ("C"), the LCD would display "CC" even when the direction was Clockwise. This was because the single character "C" did not overwrite the second "C" from the previous state.

Solution: The display logic was updated to print a trailing space (`lcd.print("C ")`) when in Clockwise mode. This explicit space overwrites the leftover character from the "CC" string, ensuring the display accurately reflects the current state.

C. Motor Jitter and Instability

Problem: Initially, the motor speed fluctuated wildly even in a quiet room. This was caused by using a single `analogRead()` execution, which captures instantaneous noise on the AC audio wave.

Solution: A sampling window algorithm was implemented. The code now samples the audio pin for 50 milliseconds to find the signal's maximum and minimum peaks. The difference (`signalMax - signalMin`) provides a stable representation of loudness.

III. TECHNICAL DETAILS

As the sole contributor to this project, I was responsible for all aspects of the design and implementation.

Hardware Design and Assembly:

- Wired the L293D motor driver, connecting ENABLE to Pin 5 and Directions to Pins 3 and 4.
- Connected the LCD 1602 module in 4-bit parallel mode to Pins 7-12 and tuned the contrast potentiometer for visibility.
- Configured the DS1307 RTC on the Mega's I2C bus.
- Physically assembled and wired the entire circuit on a breadboard.

Software Design (Arduino):

- Implemented Timer Interrupts (Timer1) to handle timekeeping asynchronously, meeting the specific lab requirement to retrieve RTC data only once at startup using `clock.getDateTime()`.
- Designed the Audio Sampling Window to convert raw analog waveforms into usable amplitude data.
- Utilized the DS3231 library to interface with the DS1307 hardware, using the shared I2C register map for timekeeping functions.
- Implemented software debouncing (`delay(50)`) inside the button logic to ensure clean toggling of the `isCW` state.

Debugging and Problem Solving:

- Debugged the entire system from end-to-end. This included using the serial monitor to trace program flaws.
- Used the I2C Scanner sketch to diagnose the initial RTC wiring fault.

IV. CONCLUSION

This lab provided comprehensive exercise in embedded system integration, combining time-sensitive interrupts with analog signal processing. A primary lesson was the distinction between polling and interrupts; utilizing the internal Timer1 allowed for precise timekeeping without requiring the RTC's SQW pin. Another critical takeaway was the handling of "noisy" analog data. Simple polling is insufficient for audio; a sampling window approach is required to extract meaningful amplitude data. Finally, this project reinforced the importance of state management in the display logic—specifically, ensuring that old data is properly cleared from an LCD when new, shorter data is written.

V. REFERENCES

- [1] *Lab 3: Audio-Driven Kinetic Sculpture with Infrared Controle*, Luo.Y, 2025
- [2] *eece-4520-5520-labs*, Luo.Y, GitHub, 2025. <https://github.com/ACANETS/eece-4520-5520-labs>
- [4] *AVR® Instruction Set Manual*, Microchip Technology Inc, 2021.
- [5] “Arduino mega2560 re v3.” Snapeda. <https://www.snapeda.com/parts/ATmega2560>
- [6] *Elegoo mega 2560 R3*, ELEGOO, 2025.
- [7] *DS1307 64 X 8 Serial Real Time Clock*, Dallas Semiconductor, 2025.
- [8] *RTCLib/DS3231 Library*, Jarzebski, GitHub. <https://github.com/jarzebski/Arduino-DS3231.git>
- [9] *LiquidCrystal Library Documentation*, Arduino Reference.
- [10] *Large Microphone Module*, ELEGOO, 2025.